

AI Engineering & Model Development Track

A Capability Building Proposal for EXL Service by Ambilio

We are pleased to present this advanced, hands-on training program for AI, ML, and GenAI Engineers — built for 2026 production realities. Every module focuses on what engineers actually build, ship, and operate, from LLM integration and RAG systems to fine-tuning and LLMOps.

	Training Outline			Duration
	#	Module	What is to be covered?	
Pre-ILT Readings	0	Pre-Readings with LMS	Python basics: functions, loops, classes, and file I/O, NumPy and Pandas — data manipulation fundamentals, Linear algebra basics: vectors, matrices, dot products, How neural networks learn: weights, loss, and backpropagation, Using APIs: HTTP requests, JSON, authentication, and environment variables	3 Hours, Self-Paced
Instructor-Led Training (ILT) Workshops	1	Python & Engineering Foundations for AI Systems	Async Python — asyncio for high-throughput AI workloads, Pydantic for data validation, schema enforcement, and output parsing, FastAPI — building production-grade AI inference endpoints, Designing modular, testable AI application code structure, Working with LLM SDKs: OpenAI, Anthropic, Gemini, HuggingFace, Environment management, secrets handling, and config best practices, Git-based workflow for AI codebases — branching, versioning, PR standards	35 Hours
	2	Machine Learning Engineering — Applied Production Skills	Feature engineering and preprocessing pipelines with Scikit-learn, Supervised and unsupervised models — when LLMs are overkill, Experiment tracking with MLflow and Weights & Biases, Hyperparameter tuning — Optuna, grid search, early stopping, Model evaluation: precision, recall, F1, AUC — beyond accuracy, Handling class imbalance, data drift, and distribution shift, Packaging and serving classical ML models via REST APIs	
	3	Deep Learning & Neural Network Engineering	PyTorch — tensors, autograd, forward/backward pass hands-on, Transformer architecture: attention, tokenization, positional encoding, Building and training neural networks from scratch in PyTorch, CNNs for vision, LSTMs vs. Transformers for sequences, GPU memory management and mixed-precision training (FP16/BF16), Overfitting controls: dropout, weight decay, early stopping, Model checkpointing, saving, loading, and ONNX export	
	4	LLM Engineering — Working with Foundation Models	LLM API integration: OpenAI, Anthropic, Gemini, Bedrock, Vertex AI, Token budgeting, cost-per-call modeling, and latency profiling, Context window management — chunking, summarization, sliding window, Structured output engineering — JSON mode, function calling, tool use, Prompt versioning and regression testing pipelines, Streaming responses, async calls, and retry/fallback handling, Model routing — sending tasks to cheapest capable model dynamically	
	5	RAG Systems — Build and Optimize End-to-End	Document ingestion pipelines — parsing PDFs, HTML, tables, images, Chunking strategies: fixed, semantic, recursive, late chunking, Embedding models: OpenAI, Cohere, BGE, Jina — selection and benchmarking, Vector databases: Pinecone, Qdrant, Weaviate, pgvector — setup and operations, Hybrid search: BM25 + dense vector + Reciprocal Rank Fusion, Re-ranking with cross-encoders (Cohere Rerank, BGE-Reranker), RAG evaluation with RAGAS: faithfulness, answer relevance, context recall	
	6	Fine-Tuning, Alignment & Model Customization	When to fine-tune vs. prompt vs. RAG — decision framework, Supervised Fine-Tuning (SFT) — dataset preparation, quality over quantity, LoRA and QLoRA — adapter-based training on consumer and cloud GPUs, DPO (Direct Preference Optimization) — aligning model outputs to standards, Synthetic data generation for fine-tuning without labeled datasets, Fine-tuning with HuggingFace TRL, Unsloth, and LLaMA-Factory, Evaluating fine-tuned models: MT-Bench, MMLU, LLM-as-Judge	
	7	AI Agent Engineering	Agent anatomy: tools, memory, reasoning loop, output validation, LangGraph — stateful, cyclical workflows with checkpointing, Tool calling: designing, registering, and testing tool functions, Memory architecture: in-context vs. external (Redis, Postgres, pgvector), Multi-agent systems with CrewAI and AutoGen — roles and handoffs, MCP integration — connecting agents to internal APIs and data sources, Error handling, loop detection, and retry logic in agentic workflows	

	8	Multimodal AI Engineering	Vision-language models: GPT-5 and 4o, Gemini 2.5, LLaVA, Qwen — integration patterns, Document AI: extracting structured data from PDFs, forms, invoices, Image-to-text pipelines for enterprise document processing, Audio processing: Whisper for transcription, speech-to-agent workflows, Multimodal RAG — embedding and retrieving across text and image, Building multimodal pipelines with LangChain and LlamaIndex, Handling multi-format inputs in a single inference endpoint	
	9	LLMOps — Deploying and Operating AI in Production	Model serving with vLLM and TGI — throughput, batching, concurrency, Containerizing AI applications with Docker and deploying on Kubernetes, CI/CD pipelines for LLM applications — automated eval before every deploy, LLM observability with LangSmith, Phoenix/Arize, and Helicone, Monitoring: latency (TTFT), token usage, cost drift, and output quality, Prompt versioning, A/B testing, and shadow deployment for LLM apps, Semantic caching with Redis and GPTCache — reducing API call volume	
	10	AI Evaluation, Safety & Responsible Engineering	Building evaluation frameworks: task-specific test sets and scoring rubrics, LLM-as-Judge — using models to evaluate model outputs at scale, Hallucination detection and groundedness checks in production, Red-teaming: adversarial testing, jailbreak probing, edge case discovery, Implementing guardrails: NeMo Guardrails, custom input/output filters, Bias detection and fairness testing across demographic slices, Responsible AI checklist before every production deployment	
Post ILT Activities		Capstone Projects and Shark Tank Presentations	An industry relevant project involving the skills earned throughout the above modules. The participants work on project in group and present the outcomes to experts in a shark tank style presentation.	Self-Paced
		Assessment and Certification	Assessing the knowledge gained throughout the above modules. Certificates issue to those who complete all program requirements.	30 Mins

Target Audience

Suitable for early to mid-career Data and AI professional

Pre-requisites

- 1 Deep expertises in Python programing
- 2 Knowledge of AI and Machine Learning
- 3 Basic awareness of software engineering practices

Requirements

- 1 Visual Studio code and Jupyter notebook environment
- 2 Pinecone, Weviate, Chroma, neo4j

Instructors

A sample list of suggested instructors is attached separately

Major Deliverables

- 1 LMS setup with self-paced learning and resources for pre-training and post-training
- 2 Instructor-led live online training sessions
- 3 Progress analytics to track participant performance.
- 4 Capstone project allocation, mentoring, evaluation and review presentations
- 5 Assessments and certifications upon successful completion of the program